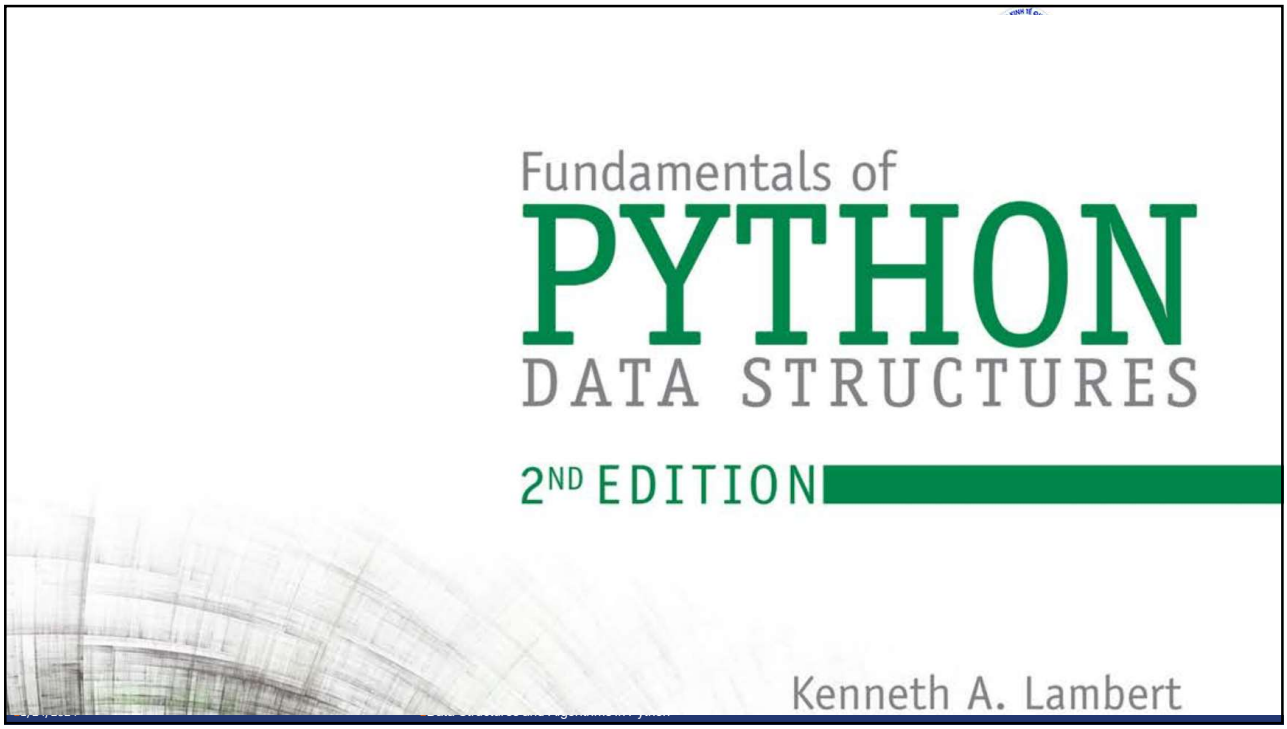

1

National Economics University
Faculty of Mathematical Economics

EP03.TITH1121 - DATA STRUCTURES AND ALGORITHMS: Stack

- Instructor: Duc Minh Vu (MFE)
- Email: minhvd [at] neu.edu.vn

2

TRƯỜNG ĐẠI HỌC KINH TẾ QUỐC DÂN
KHOA TOÁN KINH TẾ

Learning Objectives

1. Describe the features and behavior of a stack
2. Choose a stack implementation based on its performance characteristics
3. Recognize applications where it is appropriate to use a stack
4. Explain how the system call stack provides run-time support for recursive subroutines
5. Design and implement a backtracking algorithm that uses a stack

3

TRƯỜNG ĐẠI HỌC KINH TẾ QUỐC DÂN
KHOA TOÁN KINH TẾ

Overview of Stacks

- Stacks are linear collections in which access is completely restricted to just one end, called the **top**.
  - Insert an element to a collection – this new element becomes top.
  - Remove the newest element in the collection (the top)
- Adhere to a **last-in first-out (**LIFO) protocol.
- The operations for putting items on and removing items from a stack are called **push** and **pop.**

4

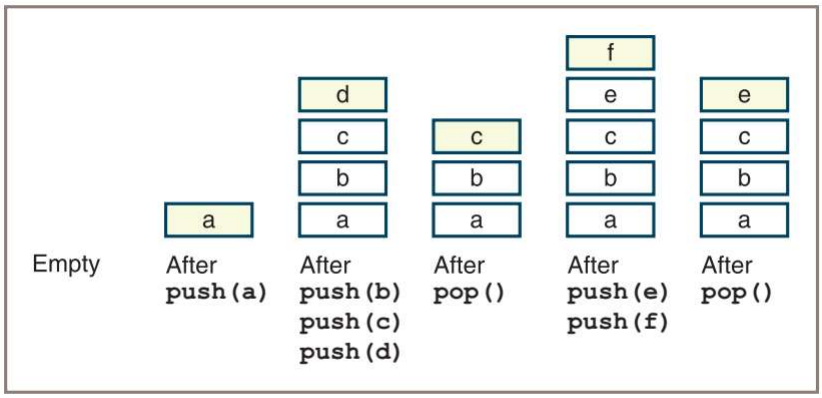

Figure 7-1 Some states in the lifetime of a stack

- Initially, the stack is empty, and then an item called **a** is pushed.
- Next, three more items called **b**, **c**, and **d** are pushed, after which the stack is popped, and so forth.

5

Some applications

- Translating infix expressions to postfix form and evaluating postfix expressions.
- Backtracking algorithms
- Managing computer memory in support of function and method calls.
- Supporting the undo feature in text editors, word processors, spreadsheet programs, drawing programs, and similar applications.
- Maintaining a history of the links visited by a web browser
- More: 3.02.Stacks.pptx (live.com)

6

Using a Stack

- The Stack Interface
  - push, pop, peek and other basic methods
    - ✓peek = examine the top element but not pop
- Instantiating a Stack
  - s1 = ArrayStack()
  - s2 = LinkedStack([20, 40, 60])

Stack Method	What It Does
`s.isEmpty()`	Returns `True` if `s` is empty or `False` otherwise.
`s.__len__()`	Same as `len(s)`. Returns the number of items in `s`.
`s.__str__()`	Same as `str(s)`. Returns the string representation of `s`.
`s.__iter__()`	Same as `iter(s)`, or `for item in s:`. Visits each item in `s`, from bottom to top.
`s.__contains__(item)`	Same as `item in s`. Returns `True` if `item` is in `s` or `False` otherwise.
`s1.__add__(s2)`	Same as `s1 + s2`. Returns a new stack containing the items in `s1` and `s2`.
`s.__eq__(anyObject)`	Same as `s == anyObject`. Returns `True` if `s` equals `anyObject` or `False` otherwise. Two stacks are equal if the items at corresponding positions are equal.
`s.clear()`	Makes `s` become empty.
`s.peek()`	Returns the item at the top of `s`. *Precondition:* `s` must not be empty; raises a `keyerror` if the stack is empty.
`s.push(item)`	Adds `item` to the top of `s`.
`s.pop()`	Removes and returns the item at the top of `s`. *Precondition:* `s` must not be empty; raises a `KeyError` if the stack is empty.

Table 7-1 The Methods in the Stack Interface

7

TRƯỜNG ĐẠI HỌC KINH TẾ QUỐC DÂN
KHOA TOÁN KINH TẾ

Operation	State of the Stack After the Operation	Value Returned	Comment
`s = <Stack Type>()`			Initially, the stack is empty.
`s.push(a)`	`a`		The stack contains the single item `a`.
`s.push(b)`	`a b`		`b` is the top item.
`s.push(c)`	`a b c`		`c` is the top item.
`s.isEmpty()`	`a b c`	`False`	The stack is not empty.
`len(s)`	`a b c`	`3`	The stack contains three items.
`s.peek()`	`a b c`	`c`	Return the top item without removing it.
`s.pop()`	`a b`	`c`	Remove and return the top item. `b` is now the top item.
`s.pop()`	`a`	`b`	Remove and return the top item. `a` is now the top item.
`s.pop()`		`a`	Remove and return the top item.
`s.isEmpty()`		`True`	The stack is empty.
`s.peek()`		`KeyError`	Peeking at an empty stack raises an exception.
`s.pop()`		`KeyError`	Popping an empty stack raises an exception.
`s.push(d)`	`D`		`d` is the top item.

Table 7-2 The Effects of Stack Operations

8

Example Application: Matching Parentheses

- Determine if the bracketing symbols in expressions are balanced correctly.

Example Expression	Status	Reason
(...)...(...)	Balanced	
(...)...(...	Unbalanced	Missing a closing) at the end.
)...(...(...)	Unbalanced	The closing) at the beginning has no matching opening (and one of the opening parentheses has no closing parenthesis.
[...(...)...]	Balanced	
[...(...]...)	Unbalanced	The bracketed sections are not nested properly.

Table 7-3 Balanced and Unbalanced Brackets in Expressions

9

TRƯỜNG ĐẠI HỌC KINH TẾ QUỐC DÂN
KHOA TOÁN KINH TẾ

Example Application: Matching Parentheses

- Parenthesis is balanced if we can find a right match for each (open) parenthesis
  - We maintain a collection of parentheses that have not found matchings yet
  - Traverse from left to right:
    - ✓If the current bracket is an open bracket : push into the collection
    - ✓If the current bracket is a close bracket: check whether the newest open bracket matches the current close bracket
      - Remove/Pop if they are matching
      - Otherwise, declare unbalanced parenthesis
  - If the collection is not empty after checking every bracket: return unbalanced parenthesis (more open parenthesis than close open thesis) – otherwise, return true.

10

```
"""
File: brackets.py
Checks expressions for matching brackets
"""
from linkedstack import LinkedStack
def bracketsBalance(exp):
    """exp is a string that represents the expression"""
    stk = LinkedStack()                          # Create a new stack
    for ch in exp:                           # Scan across the expression
        if ch in ['[', '(']:                   # Push an opening bracket
            stk.push(ch)

        elif ch in [']', ')']:               # Process a closing bracket
            if stk.isEmpty():                          # Not balanced
                return False
            chFromStack = stk.pop()
            # Brackets must be of same type and match up
            if ch == ']' and chFromStack != '[' or \
                ch == ')' and chFromStack != '(':
                return False
    return stk.isEmpty()                             # They all matched up

def main():
    exp = input("Enter a bracketed expression: ")
    if bracketsBalance(exp):
        print("OK")
    else:
        print("Not OK")
if __name__ == "__main__":
    main()
```


11

Exercises

1. Using the format of Table 7-2, complete a table that involves the following sequence of stack operations.

Operation
s = <Stack Type>()
s.push(a)
s.push(b)
s.push(c)
s.pop()
s.pop()
s.peek()
s.push(x)
s.pop()
s.pop()
s.pop()

The other columns are labeled **State of the Stack After the Operation**, **Value Returned**, and **Comment**.

12

2. Modify the **bracketsBalance** function so that the caller can supply the brackets to match as arguments to this function. The second argument should be a list of beginning brackets, and the third argument should be a list of ending brackets. The pairs of brackets at each position in the two lists should match; that is, position 0 in the two lists might have `[` and `]`, respectively. You should be able to modify the code for the function so that it does not reference literal bracket symbols, but just uses the list arguments. (*Hint*: The method **index** returns the position of an item in a list.)

3. Someone suggests that you might not need a stack to match parentheses in expressions after all. Instead, you can set a counter to 0, increment it when a left parenthesis is encountered, and decrement it whenever a right parenthesis is seen. If the counter ever goes below 0 or remains positive at the end of the process, there was an error; if the counter is 0 at the end and never goes negative, the parentheses all match correctly. Where does this strategy break down? (*Hint*: There might be braces and brackets to match as well.)

13

TRƯỜNG ĐẠI HỌC KINH TẾ QUỐC DÂN
KHOA TOÁN KINH TẾ

Three applications of Stacks

- Evaluating Arithmetic Expressions
  - We mention only this one in the lecture
- Backtracking
- Memory Management

14

Evaluating Arithmetic Expressions

- Infix expression: each operator is located between its operands,
- Postfix expression: an operator immediately follows its operands.
- People calculate infix expressions but computer calculates postfix expression

15

TRƯỜNG ĐẠI HỌC KINH TẾ QUỐC DÂN
KHOA TOÁN KINH TẾ

Infix Form	Postfix Form	Value
34	34	34
34 + 22	34 22 +	56
34 + 22 * 2	34 22 2 * +	78
34 * 22 + 2	34 22 * 2 +	750
(34 + 22) * 2	34 22 + 2 *	112

Table 7-4 Some Infix and Postfix Expressions

16

Evaluating Postfix Expressions

- Evaluating a postfix expression involves three steps:
  - 1. Scan across the expression from left to right.
  - 2. On encountering an operator, apply it to the two preceding operands and replace all three by the result.
  - 3. Continue scanning until you reach the expression's end, at which point only the expression's value remains.
- Stack is suitable to store operands because activities in step 2 can be seen as two pops and one push

17

TRƯỜNG ĐẠI HỌC KINH TẾ QUỐC DÂN
KHOA TOÁN KINH TẾ

```
Create a new stack
While there are more tokens in the expression
   Get the next token
   If the token is an operand
        Push the operand onto the stack
   Else if the token is an operator
        Pop the top two operands from the stack
        Apply the operator to the two operands just popped
        Push the resulting value onto the stack
Return the value at the top of the stack
```

- Complexity: O(n) for postfix evaluation

18

Postfix Expression: 4 5 6 * + 3 –		Resulting Value: 31
Portion of Postfix Expression Scanned So Far	Operand Stack	Comment
		No tokens have been scanned yet. The stack is empty.
4	4	Push the operand 4.
4 5	4 5	Push the operand 5.
4 5 6	4 5 6	Push the operand 6.
4 5 6 *	4 30	Replace the top two operands by their product.
4 5 6 * +	34	Replace the top two operands by their sum.
4 5 6 * + 3	34 3	Push the operand 3.
4 5 6 * + 3 –	31	Replace the top two operands by their difference.
		Pop the final value.

Table 7-5 Tracing the Evaluation of a Postfix Expression

19

Exercises

1. Evaluate by hand the following postfix expressions:
   a. 10 5 4 + *
   b. 10 5 * 6 –
   c. 22 2 4 * /
   d. 33 6 + 3 4 / +
2. Perform a complexity analysis for postfix evaluation.

20

Converting Infix to Postfix

- Steps:
  - 1. Start with an empty postfix expression and an empty stack, which will hold operators and left parentheses.
  - 2. Scan across the infix expression from left to right.
  - 3. On encountering an operand, append it to the postfix expression.
  - 4. On encountering a left parenthesis, push it onto the stack.
  - 5. On encountering an operator, pop off the stack all operators that have equal or higher precedence, append them to the postfix expression, and then push the scanned operator onto the stack.
  - 6. On encountering a right parenthesis, shift operators from the stack to the postfix expression until meeting the matching left parenthesis, which is discarded.
  - 7. On encountering the end of the infix expression, transfer the remaining operators from the stack to the postfix expression.

21

TRƯỜNG ĐẠI HỌC KINH TẾ QUỐC DÂN
KHOA TOÁN KINH TẾ

Infix expression: 4 + 5 * 6 – 3		Postfix expression: 4 5 6 * + 3 –	
Portion of Infix Expression So Far	**Operator Stack**	**Postfix Expression**	**Comment**
			No tokens have been seen yet. The stack and the PE are empty.
4		4	Append 4 to the PE.
4 +	+		Push + onto the stack.
4 + 5	+	4 5	Append 5 to the PE.
4 + 5 *	+ *	4 5	Push * onto the stack.
4 + 5 * 6	+ *	4 5 6	Append 6 to the PE.
4 + 5 * 6 –	–	4 5 6 * +	+ Pop * and +, append them to the PE, and push – onto the stack.
4 + 5 * 6 – 3	–	4 5 6 * + 3	Append 3 to the PE.
4 + 5 * 6 – 3		4 5 6 * + 3 –	Pop the remaining operators off the stack and append them to the PE.

Table 7-6 Tracing the Conversion of an Infix Expression to a Postfix Expression

22

Infix expression: (4 + 5) * (6 − 3)		Postfix expression: 4 5 + 6 3 − *	
Portion of Infix Expression So Far	**Operator Stack**	**Postfix Expression**	**Comment**
			No tokens have been seen yet. The stack and the PE are empty.
(	(		Push (onto the stack.
(4		4	Append 4 to the PE.
(4 +	(+		Push + onto the stack.
(4 + 5	(+	4 5	Append 5 to the PE.
(4 + 5)		4 5 +	Pop the stack until (is encountered, and append operators to the PE.
(4 + 5) *	*	4 5 +	Push * onto the stack.
(4 + 5) * (	* (	4 5 +	Push (onto the stack.
(4 + 5) * (6	* (	4 5 + 6	Append 6 to the PE.
(4 + 5) * (6 −	* (−	4 5 + 6	Push - onto the stack.
(4 + 5) * (6 − 3	* (−	4 5 + 6 3	Append 3 to the PE.
(4 + 5) * (6 − 3)	*	4 5 + 6 3 −	Pop the stack until (is encountered, and append operators to the PE.
(4 + 5) * (6 − 3)		4 5 + 6 3 − *	Pop the remaining operators off the stack, and append them to the PE.

Table 7-7 Tracing the Conversion of an Infix Expression to a Postfix Expression

23

Exercises

1. Translate by hand the following infix expressions to postfix form:
   a. 33 − 15 * 6
   b. 11 * (6 + 2)
   c. 17 + 3 − 5
   d. 22 − 6 + 33 / 4
2. Perform a complexity analysis for a conversion of infix to postfix.

24

Implementations of Stacks

- Stacks are implemented easily using arrays or linked structures

25


```
"""
File: teststack.py
Author: Ken Lambert
A tester program for stack implementations.
"""

from arraystack import ArrayStack
from linkedstack import LinkedStack

def test(stackType):
      # Test any implementation with the same code
      s = stackType()
      print("Length:", len(s))
      print("Empty:", s.isEmpty())
      print("Push 1-10")
      for i in range(10):
            s.push(i + 1)
      print("Peeking:", s.peek())
      print("Items (bottom to top):", s)
      print("Length:", len(s))
      print("Empty:", s.isEmpty())
      theClone = stackType(s)
      print("Items in clone (bottom to top):", theClone)
      theClone.clear()
      print("Length of clone after clear:", len(theClone))
      print("Push 11")
      s.push(11)
      print("Popping items (top to bottom):", end = " ")
      while not s.isEmpty(): print(s.pop(), end = " ")
      print("\nLength:", len(s))
      print("Empty:", s.isEmpty())

# test(ArrayStack)
test(LinkedStack)
```


26

Array Implementation

```
"""
File: arraystack.py
"""

from arrays import Array
from abstractstack import AbstractStack

class ArrayStack(AbstractStack):
    """An array-based stack implementation."""

    DEFAULT_CAPACITY = 10          # For all array stacks

    def __init__(self, sourceCollection = None):
        """Sets the initial state of self, which includes the
        contents of sourceCollection, if it's present."""
        self.items = Array(ArrayStack.DEFAULT_CAPACITY)
        AbstractStack.__init__(self, sourceCollection)

        self.items = Node(item, self.items)
        self.size += 1

    def pop(self):
        """Removes and returns the item at top of the stack.
        Precondition: the stack is not empty."""
        if self.isEmpty():
            raise KeyError("The stack is empty.")
        oldItem = self.items.data
        self.items = self.items.next
        self.size -= 1
        return oldItem
```

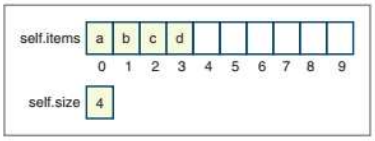

Figure 7-6 An array representation of a stack with four items

27

Linked Implementation

```
from node import Node
from abstractstack import AbstractStack

class LinkedStack(AbstractStack):
    """ Link-based stack implementation."""

    def __init__(self, sourceCollection = None):
        self.items = None
        AbstractStack.__init__(self, sourceCollection)

    # Accessors
    def __iter__(self):
        """Supports iteration over a view of self.
        Visits items from bottom to top of stack."""
        def visitNodes(node):
            if node != None:
                visitNodes(node.next)
                tempList.append(node.data)
        tempList = list()
        visitNodes(self.items)
        return iter(tempList)

    def peek(self):
        """Returns the item at top of the stack.
        Precondition: the stack is not empty."""
        if self.isEmpty():
            raise KeyError("The stack is empty.")
        return self.items.data
```

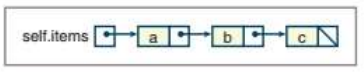

Figure 7-7 A linked representation of a stack with three items

28

Linked Implementation

```
# Mutators
def clear(self):
    """Makes self become empty."""
    self.size = 0
    self.items = None

def push(self, item):
    """Inserts item at top of the stack."""

    self.items = Node(item, self.items)
    self.size += 1

def pop(self):
    """Removes and returns the item at top of the stack.
    Precondition: the stack is not empty."""
    if self.isEmpty():
        raise KeyError("The stack is empty.")
    oldItem = self.items.data
    self.items = self.items.next
    self.size -= 1
    return oldItem
```

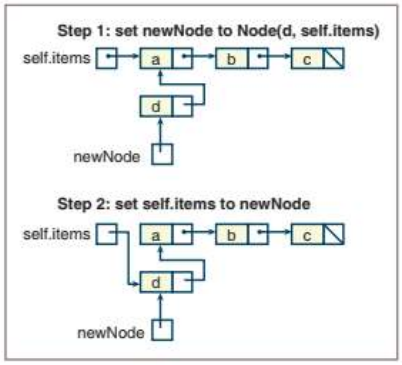

Figure 7-8 Pushing an item onto a linked stack

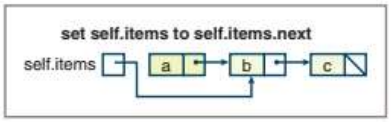

Figure 7-9 Popping an item from a linked stack

29

Exercises

1. Discuss the difference between using an array and using a Python list to implement the class **ArrayStack**. What are the trade-offs?
2. Add code to the methods **peek** and **pop** in **ArrayStack** so that they raise an exception if their preconditions are violated.
3. Modify the method **pop** in **ArrayStack** so that it reduces the capacity of the array if it is underutilized.

30

Review Questions

1. Examples of stacks are:
   a. Customers waiting in a checkout line
   b. A deck of playing cards
   c. A file directory system
   d. A line of cars at a tollbooth
   e. Laundry in a hamper
2. The operations that modify a stack are called:
   a. Add and remove b. Push and pop
3. Stacks are also known as:
   a. First-in first-out data structures
   b. Last-in, first-out data structures
4. The postfix equivalent of the expression 3 + 4 * 7 is:
   a. 3 4 + 7 * b. 3 4 7 * +
5. The infix equivalent of the postfix expression 22 45 11 * − is:
   a. 22 − 45 * 11 b. 45 * 11 − 22
6. The value of the postfix expression 5 6 + 2 * is:
   a. 40 b. 22
7. Memory for function or method parameters is allocated on the:
   a. Object heap b. Call stack
8. The running time of the two stack-mutator operations is:
   a. Linear b. Constant
9. The linked implementation of a stack uses nodes with:
   a. A link to the next node
   b. Links to the next and previous nodes
10. The array implementation of a stack places the top element at the:
    a. First position in the array
    b. Position after the last element that was inserted

31